

《SmartMail 智能邮件系统项目计划书》

作者：李明轩

完成日期：2026/6/6

签收人：

签收日期：

修改情况记录：

版本号	修改批准人	修改人	签收人	修改说明
V0.1	项目组	李明轩		根据课程要求和初始草稿形成计划书草案
V1.0	项目组	李明轩		结合当前仓库 MVP、架构文档、API 文档和 Agent 实现完善正式版

目录

1. 项目背景、范围及目标
2. 项目可行性分析
3. 项目实施计划
4. 项目任务分解结构（WBS）

1. 项目背景、范围及目标

1.1 项目背景

SmartMail 智能邮件系统是依据 2026 年软件开发综合实训“邮件系统”题目要求规划和开发的 Web 应用项目。课程要求系统至少具备邮箱注册、用户登录、发送邮件到服务器、从服务器接收或拉取邮件、服务器持久化邮件数据等基础能力，同时要求邮件正文和附件分离存储，并鼓励在基础邮件服务器中以可插拔方式融合智能系统能力。

传统邮件系统通常依赖 SMTP、POP3 或 IMAP 协议，但实训要求允许基于 HTTP、WebSocket 或 TCP 自定义上层协议实现邮件收发。结合团队开发周期、成员技术栈、前后端协同成本和答辩展示效果，本项目采用 Web 版前后端分离架构，以 HTTP REST API

作为首版通信协议，在同一服务器用户之间实现邮件收发闭环，并通过独立 Python Agent 服务提供邮件摘要、回复草稿、优先级分析和标签建议等智能能力。

截至本计划书编制时，项目仓库已完成第一轮最小 MVP：前端、后端、Agent 三部分均已搭建 mock 骨架；后端已建立 Spring Boot、数据库 schema、基础实体、Mapper、认证、邮件收发、邮箱列表、邮件详情、AI 门面接口和 Internal Tool API；Agent 已完成 FastAPI 服务、任务接口、规则版 Agent Loop、后端工具调用和 mock fallback；前端已完成类 Notion Mail 的工作台骨架，可调用后端 API，也可在后端不可用时使用 mock 数据。

1.2 项目意义

本项目不是单纯实现一个页面演示型邮箱，而是围绕“可运行的业务系统 + 可插拔智能能力 + 可协作工程结构”三个目标展开。项目的实践意义主要体现在：

1. 完整覆盖全栈开发流程：前端 Vue、后端 Spring Boot、数据库 MySQL、Python Agent 和接口联调均纳入统一工程。
2. 训练系统架构设计能力：通过前后端分离、业务后端和 Agent 解耦、Internal API、安全边界和数据分层设计，形成可解释、可扩展的系统结构。
3. 满足课程对数据库设计的要求：邮件主数据、收件人关系、用户邮箱视图、AI 结果等拆分为多张表，避免把复杂业务压缩到单表中。
4. 支持智能系统加分项：Agent 以服务化、可开关、可降级方式接入，不影响基础邮件系统可用性。
5. 便于团队并行开发：前端、后端、Agent 和文档分别有清晰目录和职责边界，适合 6 人小组分模块推进。

1.3 项目范围

1.3.1 首版范围

首版 SmartMail 的范围是“同一服务器用户之间的 Web 邮件系统 + 基础智能邮件处理能力”。系统假设所有用户均注册在同一服务器中，不处理跨邮件服务器转发，不接入真实外部邮箱，也不优先实现完整 SMTP、POP3、IMAP 协议栈。

首版包含以下范围：

- 用户注册、登录、退出和 Token 鉴权。
- 发送邮件、收取邮件、拉取邮箱列表、查看邮件详情。
- 收件箱、已发送、垃圾箱等基础邮箱视图。
- 已读/未读、星标、删除到垃圾箱、软删除等状态管理。
- 邮件正文入库，附件文件落盘，附件元信息入库。
- AI 邮件摘要、回复草稿、优先级分析、标签建议和垃圾邮件风险提示。
- 后端 AI 门面接口和 Agent Internal Tool API。
- 前端类 Notion Mail 的工作台式交互界面。
- 项目计划、接口、设计、测试和报告文档。

1.3.2 暂不纳入首版范围

为保证实训周期内交付稳定，本项目首版暂不承诺以下内容：

- 不实现跨邮件服务器通信。
- 不直接对接真实 SMTP/POP3/IMAP 服务器。
- 不实现企业级全文搜索集群或向量数据库。
- 不实现复杂多租户、组织通讯录和管理员后台。
- 不把 AI 服务作为基础邮件功能的强依赖。
- 不在首版引入 Kubernetes 等重型部署方式。

上述能力可作为第二阶段扩展或答辩中的演进设计说明。

1.4 系统总体架构

系统采用三服务架构：

层级	技术	职责
前端	Vue 3、Vite、TypeScript、Element Plus、Pinia、Axios	邮件工作台、登录注册、邮箱列表、详情阅读、写信、AI 面板
主业务后端	Spring Boot 3、Java 17、MyBatis-Plus、MySQL 8、Token 鉴权	用户、邮件、邮箱状态、附件、搜索、标签、AI 门面、Internal Tool API
AI Agent	Python、FastAPI、Pydantic、httpx、规则引擎、LLM SDK 预留	邮件摘要、回复草稿、优先级分析、标签建议、工具调用和 AI 结果写回

逻辑链路如下：

前端 Vue 工作台调用 Spring Boot REST API；Spring Boot 负责业务规则和数据库持久化；当用户触发 AI 任务时，后端调用 Python Agent；Agent 通过 Internal Tool API 获取邮件上下文、写回 AI 结果、更新邮件优先级。Agent 不直连数据库，前端也不直接调用 Agent。

1.5 系统基本功能

1.5.1 基础邮件功能

功能模块	功能说明
用户认证	邮箱注册、登录、Token 签发、当前用户查询
邮件发送	发件人填写收件人、抄送、主题、正文和附件，后端保存邮件并生成邮箱条目
邮件接收	收件人通过收件箱接口拉取邮件，后续可扩展 SSE 或 WebSocket 新邮件通知
邮箱管理	收件箱、已发送、垃圾箱、已读/未读、星标、删除
附件管理	附件上传、下载、磁盘隔离存储、数据库保存元信息
搜索与筛选	按主题、正文、发件人、文件夹、已读状态和时间范围查询
标签管理	用户自定义标签、AI 标签建议、邮件标签关系

1.5.2 智能邮件功能

智能能力	实现方式	价值
邮件摘要	Agent 获取邮件上下文后生成结构化摘要和行动项	降低阅读长邮件成本
回复草稿	根据邮件主题、发件人和正文生成可编辑回复	提高邮件处理效率
优先级分析	基于关键词、任务紧急性、风险词等规则判断优先级	帮助用户优先处理重要邮件
标签建议	识别课程、会议、附件、待处理等标签	改善邮件归类和检索体验
垃圾邮件识别	首版规则识别，后续可接入 LLM 或机器学习模型	增强邮件安全性

1.6 要解决的核心问题

问题	设计方案	预期效果
邮件原文和用户状态混杂	`mail_message` 保存邮件主数据, `mailbox_item` 保存用户视图状态	支持不同用户独立已读、删除、星标和优先级
附件不适合直接入库	文件存磁盘, 数据库保存路径、文件名、MIME、大小和校验信息	降低数据库压力, 符合课程要求
AI 服务不稳定可能影响主链路	后端提供 AI 门面和降级返回, Agent 可选启动	AI 失效时基础邮件功能仍可用
前后端联调周期短	先定义 REST API 和 mock 数据, 再逐步切换真实接口	降低协作阻塞
Agent 可能越权访问数据	Agent 不直连数据库, 只通过 Internal API 和 Internal Token 操作	控制智能系统权限边界
SMTP/POP3 状态机复杂	首版采用 HTTP REST 自定义协议	保证实训周期内可落地、可演示

1.7 项目目标

1.7.1 业务目标

- 1. 完成注册、登录、发邮件、收邮件、看详情的基础闭环。
- 2. 完成收件箱、已发送、垃圾箱和邮件状态管理。
- 3. 完成正文和附件分离存储。
- 4. 完成搜索、标签、优先级等邮件管理能力。
- 5. 完成 AI 摘要、回复草稿、优先级分析和标签建议。
- 6. 完成答辩可演示的端到端流程。

1.7.2 工程目标

- 1. 建立清晰的前端、后端、Agent、文档目录结构。

- 2. 建立接口文档、数据库设计文档和测试文档。
- 3. 采用分支开发和阶段合并，降低多人冲突。
- 4. 使主业务后端可以独立运行，AI 服务可开关、可替换。
- 5. 形成可持续扩展的项目骨架，为后续细化功能提供基础。

1.7.3 验收目标

类型	验收标准
必做功能	注册登录、发送邮件、拉取邮件、邮件详情、已读未读、删除、附件、搜索等功能可运行
数据库	表结构合理，正文入库，附件落盘，AI 结果独立存储
AI 展示	摘要、回复草稿、优先级和标签建议至少三项可演示
文档	需求、计划、设计、实现、测试、周报和答辩材料齐全
质量	主流程集成测试通过，关键接口有测试记录，演示数据完整

2. 项目可行性分析

2.1 经济可行性

SmartMail 采用开源技术栈构建，核心开发成本主要来自团队人力和时间投入。Vue、Spring Boot、MyBatis-Plus、MySQL、FastAPI、Pydantic、httpx 等均为开源技术，无需商业授权。AI 能力首版可采用规则引擎实现，后续如接入 DeepSeek、OpenAI 等大模型服务，也可通过开关控制调用频率，避免 API 成本失控。

成本项	估算	说明
开发工具	0 元	IDE、Git、Maven、Node、Python 均可免费使用
数据库	0 元	MySQL 开源免费，本地开发可使用 H2
服务器	0-100 元	实训演示可使用本机、学校服务器或轻量云主机
AI API	0-100 元	首版规则引擎可零成本；大模型调用可按需少量开启
文档工具	0 元	Markdown、Word、Draw.io 或类似工具

结论：项目经济成本低，投入主要为团队开发时间，经济可行性强。

2.2 技术可行性

本项目采用成熟、低风险、适合教学交付的技术栈。

技术	用途	成熟度	评估
Vue 3 + Vite	前端单页应用	高	构建快，生态成熟，适合邮箱工作台界面
Element Plus	UI 组件	高	表单、表格、弹窗、上传组件齐全
Spring Boot 3	主业务后端	高	企业级成熟框架，适合 REST API 和数据库业务
MyBatis-Plus	ORM/Mapper	高	适合 CRUD 密集型业务和分页查询
MySQL 8	关系数据库	高	满足用户、邮件、附件、AI 结果等结构化存储
FastAPI	Agent 服务	高	Python Web 服务开发效率高，类型提示清晰
Pydantic	Agent 数据模型	高	适合定义工具输入输出和 API schema

技术	用途	成熟度	评估
httpx	Agent 工具调用	高	适合调用 Spring Boot Internal API

当前仓库已经验证了第一轮 MVP 的技术路径：后端 `mvn test` 通过，Agent `python -m compileall app` 通过，前端 `npm run build` 通过，说明三服务工程骨架具备继续扩展的基础。

2.3 运行可行性

系统采用单机多服务方式即可运行，推荐端口如下：

服务	端口	运行方式
前端 Vite	5173	`npm run dev`
Spring Boot 后端	8080	`mvn spring-boot:run`
Python Agent	8000	`uvicorn app.main:app`
MySQL	3306	本机或容器

开发期可使用 H2 数据库启动后端，降低配置门槛；正式演示时切换 MySQL，体现数据库持久化能力。Agent 支持 mock fallback，即后端或 Internal API 暂不可用时仍可用 mock 邮件完成演示，避免因单点故障影响整体进度。

2.4 团队可行性

项目按 6 人小组设计，任务边界清晰：

角色	人数	主要职责
组长/架构师	1	需求拆解、架构设计、接口协调、集成、答辩
后端开发	2	Spring Boot、数据库、邮件收发、附件、搜索、标签
前端开发	2	Vue 工作台、列表、详情、写信、AI 面板
AI/测试	1	Agent、智能功能、测试文档、演示数据

该分工可以并行推进。后端先给出 API 契约，前端用 mock 数据开发；Agent 通过 Internal API 契约独立开发；组长负责分支合并和集成测试。

2.5 风险可行性分析

风险	概率	影响	应对措施
前后端接口不一致	中	高	先维护 `docs/api`，接口变更必须同步文档
数据库表设计返工	中	高	保持邮件原文和用户视图分离，提前评审 ER 图
Agent 接入阻塞	中	中	Agent 使用 mock fallback，后端提供稳定 Internal API
LLM 服务不稳定	中	中	首版规则引擎可独立运行，大模型作为增强项
附件安全问题	中	中	使用 UUID 路径、限制文件类型、数据库只保存受控相对路径
时间不足	中	高	严格按 MVP 优先级交付，先主流程后加分项
集成测试延期	中	高	每周至少一次合并测试，提前准备演示账号和数据

综合结论：项目经济、技术、运行和团队层面均具备可行性；主要风险集中在接口协同、数据库设计和集成节奏上，均有明确应对方案。

3. 项目实施计划

3.1 开发策略

项目采用“先闭环、后增强”的实施策略：

- 1. 第一优先级是基础邮件系统闭环，包括用户、邮件、邮箱条目和数据库持久化。
- 2. 第二优先级是前端可交互工作台，使系统可以演示真实用户流程。
- 3. 第三优先级是 Agent 和 AI 能力，先用规则版实现稳定演示，再接入 LLM。
- 4. 第四优先级是附件、搜索、标签、通知、自然语言搜索等增强项。

分支策略采用模块分支：

分支	用途
`main`	主分支，保持可演示状态
`backend/dev`	后端开发与数据库流转
`frontend/dev`	前端页面与交互
`agent/dev`	Agent、规则引擎、工具调用
`docs/dev`	文档、计划书、接口说明

3.2 阶段划分

项目计划周期为 5 周，保留第 6 周作为答辩缓冲和问题修复期。

阶段	周期	重点任务	主要交付物
阶段一：规划与架构	第 1 周	需求分析、架构设计、数据库设计、接口契约、项目骨架	计划书、架构图、ER 图、API 文档、三服务目录
阶段二：后端基础闭环	第 2 周	用户认证、邮件发送、收件箱、已发送、详情、数据库持久化	Auth/Mail/Mailbox 后端接口、schema.sql
阶段三：前端工作台与基础联调	第 3 周	登录注册、邮箱列表、邮件详情、写信、状态操作	Vue 工作台、Axios API、mock fallback
阶段四：Agent 与智能能力	第 4 周	Agent Loop、Internal Tool API、摘要、草稿、优先级、标签建议	FastAPI Agent、规则引擎、AI 结果写回
阶段五：完善与测试	第 5 周	附件、搜索、标签、集成测试、演示数据、文档补齐	测试报告、部署说明、答辩 PPT
阶段六：缓冲与答辩	第 6 周	缺陷修复、演示排练、答辩优化	最终代码、最终文档、演示脚本

3.3 详细实施计划

后端任务	前端任务	Agent/AI 任务	文档与测试
初始化 Spring Boot、设计表结构、定义 API	初始化 Vue 工程和页面原型	初始化 FastAPI 工程和数据模型	完成计划书、架构设计、接口
注册登录、Token、邮件发送、邮箱列表、详情	登录注册页、主布局、mock 邮件列表	定义 Agent task schema、mock 邮件上下文	编写 API 文档和数据库设计
已读、星标、删除、附件接口初版	邮件详情、写信、AI 面板入口	封装后端 Internal API 工具	前后端接口联调记录
AI 门面、Internal Tool API、AI 结果存储	AI 结果展示、交互优化	Agent Loop、摘要、草稿、优先级、标签建议	AI 设计文档、集成测试记录
搜索、标签、附件完善、缺陷修复	搜索筛选、附件展示、响应式优化	LLM Provider 预留或接入，规则 fallback 保留	测试计划、测试用例、测试排
部署配置、演示环境修复	UI 打磨、演示流程稳定	演示数据和 AI 输出稳定性优化	答辩 PPT、演示脚本、最终

3.4 关键里程碑

里程碑	时间	验收标准
M1 架构与接口冻结	第 1 周末	目录结构、技术选型、ER 图和 API 文档完成
M2 基础邮件闭环	第 2 周末	用户 A 能发送邮件给用户 B，用户 B 能在收件箱查看
M3 前端可演示	第 3 周末	前端能登录、查看列表、打开详情、发送邮件
M4 Agent 接入	第 4 周末	摘要、回复草稿、优先级分析可通过后端触发
M5 功能完成	第 5 周末	附件、搜索、标签、测试文档完成，主流程无阻塞缺陷
M6 答辩就绪	第 6 周	演示数据、PPT、部署说明和最终文档齐全

3.5 当前基线与后续推进

当前仓库已完成第一轮 MVP，已经具备以下基线：

- 后端：Spring Boot Maven 工程、schema、认证、邮件、邮箱、AI 门面、Internal Tool API。
- Agent：FastAPI、`/api/v1/agent/tasks`、规则版 Agent Loop、`get\_mail`、`save\_ai\_result`、`set\_priority` 工具调用。
- 前端：Vue 工作台、登录/注册、邮箱列表、阅读区、写信、AI 按钮和 mock fallback。
- 文档：MVP API、MVP 架构说明、第一轮完成记录。

后续推进应避免重新推翻架构，而是在当前边界内细化：

1. 后端补齐附件上传下载、搜索和标签 CRUD。
2. Agent 增加 LLM Provider，但保留规则引擎 fallback。
3. 前端将 `App.vue` 拆分为布局、列表、阅读器、写信、AI 面板组件。
4. 补充集成测试脚本和演示数据。
5. 把文档从“计划和接口”扩展为“设计、实现、测试、部署、使用说明”。

4. 项目任务分解结构（WBS）

4.1 WBS 分解图

SmartMail 项目 WBS 如下：

1. 项目管理与设计
 - 1.1 需求分析
 - 1.2 项目计划书
 - 1.3 系统架构设计
 - 1.4 数据库设计
 - 1.5 API 契约设计
 - 1.6 风险管理和进度管理
2. 后端开发
 - 2.1 Spring Boot 工程初始化
 - 2.2 通用响应、异常处理、配置管理
 - 2.3 用户注册登录和 Token 鉴权
 - 2.4 邮件主表和收件人关系
 - 2.5 邮箱条目和用户视图
 - 2.6 附件上传下载

- 2.7 搜索和筛选
- 2.8 标签管理
- 2.9 AI 门面接口
- 2.10 Internal Tool API
- 2.11 数据库初始化和 MySQL 配置
- 3. 前端开发
 - 3.1 Vue 3 + Vite 工程初始化
 - 3.2 登录注册页面
 - 3.3 邮件主布局
 - 3.4 邮件列表
 - 3.5 邮件详情
 - 3.6 写信和附件上传
 - 3.7 搜索筛选
 - 3.8 标签和优先级展示
 - 3.9 AI 摘要和回复草稿面板
 - 3.10 响应式和交互优化
- 4. Agent 与智能系统
 - 4.1 FastAPI Agent 工程初始化
 - 4.2 Pydantic 请求响应模型
 - 4.3 Backend Internal API 工具封装
 - 4.4 Agent Loop
 - 4.5 邮件摘要
 - 4.6 回复草稿
 - 4.7 优先级分析
 - 4.8 标签建议
 - 4.9 垃圾邮件风险识别
 - 4.10 LLM Provider 预留或接入
 - 4.11 AI 结果写回
- 5. 测试与质量保障
 - 5.1 后端接口测试
 - 5.2 前端构建测试
 - 5.3 Agent 语法和接口测试
 - 5.4 前后端联调测试
 - 5.5 后端与 Agent 集成测试
 - 5.6 演示数据准备
 - 5.7 缺陷修复和回归测试
- 6. 文档与交付
 - 6.1 需求分析文档
 - 6.2 项目计划书
 - 6.3 概要设计文档
 - 6.4 详细设计文档
 - 6.5 数据库设计文档
 - 6.6 API 文档
 - 6.7 测试文档

6.8 部署说明

6.9 答辩 PPT 和演示脚本

4.2 任务分解列表

任务名称	任务描述	负责人	工作量	达到指标	备注
需求分析	梳理课程 PDF、草稿、架构文档和开发计划，明确必做和加分项	组长/架构师	2 天	需求范围、优先级和验收标准明确	作为后续设计依据
项目计划书	编写项目背景、目标、可行性、实施计划和 WBS	组长/架构师	1 天	形成正式计划书 Word/PDF	与模板一致
架构设计	确定 Vue + Spring Boot + FastAPI 三服务架构	组长/架构师	1.5 天	输出系统架构图和模块边界	AI 与主业务解耦
数据库设计	设计用户、邮件、收件人、邮箱条目、附件、AI 结果等表	后端 1/2	2 天	ER 图和 schema.sql 通过评审	正文附件分离
API 契约	设计前后端 API、Agent API 和 Internal Tool API	架构师/后端/AI	2 天	`docs/api` 文档完成	联调基准
后端工程搭建	创建 Spring Boot Maven 工程，配置 MyBatis-Plus、MySQL/H2	后端 1	1 天	`mvn test` 通过	已完成第一轮 MVP
认证模块	实现注册、登录、Token、当前用户接口	后端 1	2 天	用户可注册登录并携带 token 调接口	后续可替换标准库
邮件发送	实现邮件主表写入、收件人关系、发送方和接收方邮箱条目	后端 2	2 天	用户 A 可发送给用户 B	核心闭环
邮箱列表与详情	实现收件箱、已发送、邮件详情查询	后端 2	2 天	邮件列表和详情可正常返回	支持分页
邮件状态	实现已读、星标、删除到垃圾箱、软删除	后端 2	1.5 天	状态变更可持久化	用户视图独立
附件模块	上传附件到磁盘，数据库保存元信息，提供下载接口	后端 1/2	2 天	附件可上传下载	下一阶段重点
搜索模块	支持主题、正文、发件人、文件夹、时间范围搜索	后端 2	2 天	`/api/v1/emails/search` 可用	先 MySQL LIKE
标签模块	实现标签 CRUD、邮件标签关系和 AI 标签落库	后端 2	1.5 天	标签可创建、绑定、筛选	配合 Agent
AI 门面	后端提供摘要、草稿、分析 API，转发 Agent	后端 2/AI	1 天	前端不直接调用 Agent	已完成 MVP
Internal API	提供 get_mail、save_ai_result、set_priority 等工具接口	后端 2/AI	1.5 天	Agent 可通过 token 调用	已完成 MVP
前端工程	初始化 Vue 3、Vite、TypeScript、Element Plus、Axios	前端 1	1 天	`npm run build` 通过	已完成 MVP
主工作台	实现侧边栏、文件夹、邮件列表、阅读区布局	前端 1	2 天	可展示 mock 和真实邮件	类 Notion Mail
登录注册	接入注册登录接口和 token 存储	前端 1	1.5 天	可登录并进入工作台	需补路由守卫
写信功能	实现收件人、抄送、主题、正文、发送按钮	前端 2	1.5 天	可调用发送接口	后续补附件
邮件详情	展示正文、收件人、状态和 AI 结果	前端 2	2 天	用户能阅读邮件	已有 MVP 骨架
搜索筛选	实现搜索框、高级筛选和结果展示	前端 2	2 天	与后端搜索接口联调	下一阶段
AI 面板	展示摘要、草稿、优先级、标签建议	前端 2/AI	1.5 天	AI 结果可读、可刷新	已有入口
Agent 工程	初始化 FastAPI、Pydantic、配置和路由	AI/测试	1 天	`/health` 和 `/tasks` 可用	已完成 MVP
工具调用封装	使用 httpx 封装后端 Internal API	AI/测试	1.5 天	可 get_mail、save_ai_result、set_priority	已完成 MVP
Agent Loop	实现 plan、tool、reason、save 的轻量流程	AI/测试	2 天	返回结构化 steps 和 result	已完成 MVP
摘要功能	根据邮件正文生成摘要和行动项	AI/测试	1 天	summary 任务可运行	规则版已完成
回复草稿	根据发件人、主题和正文生成可编辑回复	AI/测试	1 天	reply_draft 任务可运行	规则版已完成
优先级与标签	根据关键词识别优先级、标签和风险提示	AI/测试	1.5 天	analyze 任务可运行并写回	已完成 MVP
LLM 接入	增加 OpenAI/DeepSeek Provider 和提示词模板	AI/测试	2 天	可配置 API Key 后切换大模型	加分项
后端测试	编写认证、邮件、邮箱、AI 门面接口测试	后端 1/2	2 天	主接口测试通过	目标覆盖关键路径
Agent 测试	测试三类任务、工具调用失败 fallback、结果写回	AI/测试	1.5 天	Agent 异常可控	重点保障
前端测试	构建测试、关键交互测试、接口错误提示	前端 1/2	1.5 天	构建通过，主页面可用	可手工测试
集成测试	注册、发送、接收、详情、AI 分析全流程	全体	2 天	主流程端到端通过	答辩前必须完成
演示数据	准备多用户、多邮件、附件、标签、AI 结果	AI/测试	1 天	演示稳定可复现	答辩关键
设计文档	完成概要设计、详细设计、数据库设计、API 文档	组长/各模块	3 天	文档完整且与代码一致	课程提交物

任务名称	任务描述	负责人	工作量	达到指标	备注
测试文档	完成测试计划、测试用例、测试报告	AI/测试	2 天	覆盖主流程和异常流	课程提交物
部署说明	编写三服务启动、环境变量、数据库初始化说明	后端 1/组长	1 天	他人可按文档启动	交付保障
答辩材料	制作 PPT、演示脚本、亮点说明	组长/全体	2 天	10 分钟内清晰展示系统	突出 AI 和架构

4.3 角色职责

角色	主要职责	关键交付
组长/架构师	需求拆解、架构设计、接口协调、风险管理、最终集成	计划书、架构图、接口评审、答辩组织
后端 1	认证、安全、通用模块、附件、部署配置	Auth、User、Common、Attachment、部署文档
后端 2	邮件主链路、邮箱状态、搜索、标签、AI 门面	Mail、Mailbox、Search、Label、AI facade
前端 1	前端工程、路由、状态管理、主布局、登录注册	Vue 工程、主布局、认证页面
前端 2	邮件列表、详情、写信、附件、AI 面板	邮件工作台核心页面
AI/测试	Agent、规则引擎、LLM 接入、测试文档、演示数据	FastAPI Agent、测试报告、演示数据

4.4 质量保障计划

质量活动	执行方式	输出
接口评审	每次新增接口先更新 `docs/api`，前后端确认后开发	API 文档
数据库评审	核心表变更前评审字段、索引、关系和影响范围	ER 图和 SQL
分支开发	前端、后端、Agent、文档分别分支开发，阶段性合并 main	清晰提交历史
构建检查	后端 `mvn test`，前端 `npm run build`，Agent `compileall`	构建记录
集成测试	三服务同时启动，跑注册、发送、接收、AI 分析流程	集成测试记录
演示检查	固定演示账号和演示邮件，提前排练答辩流程	演示脚本

4.5 验收标准

类别	验收标准
功能完整性	必做邮件功能全部可运行，AI 展示功能至少三项可演示
架构合理性	前端、后端、Agent 边界清晰，Agent 不直连数据库
数据合理性	多表设计符合邮件业务，正文附件分离，AI 结果单独存储
可运行性	按部署文档可启动三服务并完成主流程
可维护性	模块命名清晰，接口文档同步，代码结构可继续扩展
答辩表现	能说明需求、架构、数据库、核心流程、AI 插件化设计和风险控制

4.6 计划结论

SmartMail 项目采用 Web 前后端分离、Spring Boot 主业务后端和 Python Agent 智能服务的架构，符合课程对邮件系统、数据库设计、正文附件分离和智能插件的要求。当前仓库已完成第一轮 MVP，为后续功能细化提供了稳定基线。后续工作应围绕附件、搜索、标签、LLM 接入、测试和答辩材料展开，确保先交付稳定主流程，再展示智能增强能力。

从项目架构角度看，本计划的关键不是追求协议或 AI 框架复杂度，而是在实训周期内构建一个边界清晰、能运行、能演示、能解释的系统。该路线技术风险可控，团队分工明确，成果可验证，适合作为 SmartMail 项目的正式实施计划。